Civil Engineering Analysis II – Project#5 Annotated Coding Document

Transportation 01 Group 17

Nicole Centeno, Sofia Lopez, Johnathan Sevick

05/08/2026

Python Code	Annotation
import pandas as pd import numpy as np import nbformat import ast import re	Importing of various libraries with import function
import matplotlib.pyplot as plt from matplotlib.patches import Polygon as MplPolygon import matplotlib.animation as animation from IPython.display import HTML	This block imports matplotlib for creating animated polygon visualizations and IPython to display them inline inside a jupyter notebook
pd.set_option("display.max_columns", None) pd.set_option("display.width", 200)	These two lines tell pandas to display all columns without truncation and set the output width to 200 characters
data_file = "Transformed_TGSIM_Foggy_Bottom_200sec(in).csv" polygon_file = "Polygons.ipynb"	These two define variables for the foggy bottom data and polygon definitions
dataset_summary_file = "dataset_description_summary.csv" group_summary_file = "agent_group_summary.csv" decel_agent_file = "deceleration_by_agent.csv" decel_group_file = "deceleration_by_group.csv" lane_summary_file = "lane_speed_summary.csv"	These lines define variables for five CSV output files that will store various summaries and analyses. (dataset description, deceleration by group and agent and lane speed)
df = pd.read_csv(data_file) df.columns = df.columns.str.strip()	These two load the csv with pandas .read_csv and then strip away whitespace
df = df.rename(columns={"x-coord(m)": "xloc_kf","y-coord(m)": "yloc_kf","lane_id": "lane_kf", "x-speed(m/s)": "speed_kf_x","y-speed(m/s)": "speed_kf_y","x-accel(m/s2)": "acceleration_kf_x", "y-accel(m/s2)": "acceleration_kf_y",})	This line renames several DataFrame columns with a consistent scheme
df = df.sort_values(["id", "time"]).reset_index(drop=True) print(df.head()) print(df.columns.tolist())	This line sorts the DataFrame by agent ID with .sort_values and then by time before resetting with .reset_index,

	printing the column names and first columns
type_map = {0: "Pedestrian",1: "Bicycle",2: "Scooter",3: "Passenger Car", 4: "Automated Vehicle",5: "Motorcycle",6: "Bus",7: "Truck"}	This line creates a dictionary that maps numeric agent type codes to their labels
df["agent_type"] = df["type_most_common"].map(type_map)	This line creates a new column in the DataFrame “agent_type”

def group_agent(agent_type):
if agent_type == "Automated Vehicle":
return "AV"
elif agent_type == "Passenger Car":
return "Non-AV Passenger Vehicle"
elif agent_type == "Pedestrian":
return "Pedestrian"
else:
return "Other"
	This function takes an agent type label and categorizes it into one of four groups (AV, Non-AV Passenger Vehicle, Pedestrian, or Other)

df["report_group"] = df["agent_type"].apply(group_agent) print(df["report_group"].value_counts(dropna=False))	These two lines apply the group_agent function to every row to create a new "report_group" column. Then it prints a count of each group
df["speed_total"] = np.sqrt(df["speed_kf_x"]**2 + df["speed_kf_y"]**2) df["accel_total"] = np.sqrt(df["acceleration_kf_x"]**2 + df["acceleration_kf_y"]**2)	These two lines calculate the total speed and acceleration for each agent by using the Pythagorean theorem, and stores it in two new columns
df["dt"] = df.groupby("id")["time"].diff() df["speed_change"] = df.groupby("id")["speed_total"].diff() df["signed_accel"] = df["speed_change"] / df["dt"]	These lines group the dataframe with .groupby, calculate the time difference, and speed change between consecutive rows for each agent and then divide them to get a positive or negative acceleration (speed up or slow down)
df["signed_accel"] = df["signed_accel"].replace([np.inf, -np.inf], np.nan)	This line removes any infinite values from the signed acceleration column by replacing them with NaN with .replace
df["jerk"] = df.groupby("id")["signed_accel"].diff() / df["dt"] df["jerk"] = df["jerk"].replace([np.inf, -np.inf], np.nan)	These two lines calculate jerk, (rate of change of acceleration over time) within each agent

	group and dividing by the time step, then replacing any infinite values with NaN.
print(df[["id", "time", "speed_total", "signed_accel", "jerk"]].head(10))	This line prints the first ten rows of those 5 columns to double check those calculations with .head()
dataset_summary = pd.DataFrame({ "column_name": df.columns, "data_type": [str(df[col].dtype) for col in df.columns], "non_null_count":[df[col].notna().sum() for col in df.columns], "unique_values": [df[col].nunique(dropna=True) for col in df.columns]})	This line creates a summary DataFrame with every column in the dataset, with its name, data type, count of NaN values, and number of unique values
print(dataset_summary) dataset_summary.to_csv(dataset_summary_file, index=False)	These two lines print the just made dataset summary to the console and then save it as a CSV file using the file path defined earlier.
print(f"Shape : {df.shape[0]:,} rows x {df.shape[1]} columns") print(f"Time : {df.time.min():.1f} s -> {df.time.max():.1f} s (~{df.time.max()-df.time.min():.0f} s window)") print(f"Agents : {df.id.nunique()} unique IDs") print() print("Agent type breakdown:") print(df["agent_type"].value_counts(dropna=False))	These lines print a overview of the dataset, including: total number of rows and columns, time range, number of unique agents, and how many agents exist per type.
main_df = df[df["report_group"].isin(["AV", "Non-AV Passenger Vehicle", "Pedestrian"])].copy() print(main_df["report_group"].value_counts())	These two lines filter the DataFrame to only the three main groups and print a count of each
group_summary = main_df.groupby("report_group").agg(observations = ("id", "size"), unique_agents = ("id", "nunique"), mean_speed = ("speed_total", "mean"), median_speed = ("speed_total", "median"), std_speed = ("speed_total", "std"), mean_signed_accel = ("signed_accel", "mean"), median_signed_accel = ("signed_accel", "median"), std_signed_accel = ("signed_accel", "std"), mean_accel_magnitude = ("accel_total", "mean"), median_accel_magnitude= ("accel_total", "median"), mean_jerk = ("jerk", "mean"), median_jerk = ("jerk", "median")).reset_index()	This single line groups the filtered DataFrame by group and calculates a set of summary statistics for each, including: observation counts, unique agent counts, mean, median, and standard deviation of speed, acceleration, and jerk.

print(group_summary) group_summary.to_csv(group_summary_file, index=False)	These two lines print the previous line to the console and save it as a CSV file using the file path defined earlier.
plt.figure(figsize=(9, 6)) plt.bar(group_summary["report_group"], group_summary["mean_speed"])	These two lines create a bar chart with plt.figure() and plt.bar() with report groups as the x-axis and mean speeds as the y-axis.
plt.title("Average Speed by Agent Group") plt.xlabel("Agent Group") plt.ylabel("Average Speed (m/s)") plt.grid(axis="y", alpha=0.3)	These lines give the mean speeds bar chart a title, axes labels and a semi transparent grid
plt.tight_layout() plt.show()	These two lines clean up the figure's spacing and then print the chart
plt.figure(figsize=(9, 6)) plt.bar(group_summary["report_group"], group_summary["mean_signed_accel"])	These two lines create a bar chart comparing the mean signed acceleration for each group
plt.title("Mean Signed Acceleration by Agent Group") plt.xlabel("Agent Group") plt.ylabel("Mean Signed Acceleration (m/s\u00b2)") plt.axhline(0, linewidth=1) plt.grid(axis="y", alpha=0.3)	These lines give the signed acceleration bar chart a title, axes labels and a semi transparent grid
plt.tight_layout() plt.show()	These two lines clean up the figure's spacing and then print the chart
plt.figure(figsize=(10, 6)) main_df.boxplot(column="speed_total", by="report_group", grid=False)	These two lines create a bar chart comparing the distribution of speed for each group
plt.title("Speed Distribution by Agent Group") plt.suptitle("") plt.xlabel("Agent Group") plt.ylabel("Speed (m/s)")	These lines give the speed bar chart a title, axes labels and a semi transparent grid
plt.tight_layout() plt.show()	These two lines clean up the figure's spacing and then print the chart
lane_summary = main_df.groupby(["lane_kf", "report_group"]).agg(observations = ("id", "size"),	This block groups the filtered DataFrame by both lane and

unique_agents = ("id", "nunique"), mean_speed = ("speed_total", "mean"), median_speed = ("speed_total", "median"), std_speed = ("speed_total", "std"), mean_signed_accel = ("signed_accel", "mean")).reset_index()	report group, then calculates statistics for each combination (unique agents, mean speed, median speed, standard deviation speed, mean signed acceleration)
print(lane_summary.head(20)) lane_summary.to_csv(lane_summary_file, index=False)	These two lines print the first twenty rows of the previously made summary to the console and save the full summary as a CSV file.
plt.figure(figsize=(12, 7)) for group_name in lane_summary["report_group"].dropna().unique(): temp = lane_summary[lane_summary["report_group"] == group_name].sort_values("lane_kf") plt.plot(temp["lane_kf"], temp["mean_speed"], marker="o", label=group_name)	These lines create a new figure and for loop through each report group, making a line chart of mean speed by lane for each group
plt.title("Speed Profile by Lane / Polygon ID") plt.xlabel("Lane / Polygon ID") plt.ylabel("Mean Speed (m/s)") plt.legend() plt.grid(alpha=0.3)	These lines give the previously made figure a title, axes labels and a semi transparent grid
plt.tight_layout() plt.show()	These two lines clean up the figure's spacing and then print the chart
lane_pivot = lane_summary.pivot(index="lane_kf", columns="report_group", values="mean_speed") print(lane_pivot.head(20))	These two lines create a pivot table with the lane summary with lanes as rows and report groups as columns, then print the first twenty rows to verify the result.
lane_compare = lane_pivot.copy() lane_compare["max_speed"] = lane_compare.max(axis=1) lane_compare["min_speed"] = lane_compare.min(axis=1)	These lines create a copy of the pivot table and add two new columns (highest and lowest mean speed)
lane_compare["speed_range"] = lane_compare["max_speed"] - lane_compare["min_speed"] lane_compare = lane_compare.sort_values("speed_range", ascending=False)	These two lines calculate the speed range for each lane, then sort the lanes from greatest to smallest speed variation.
print(lane_compare.head(15))	Prints the first 15 rows of the previous lines

decel_threshold = -2.0	Defines the variable for a deceleration event
main_df["hard_decel_flag"] = main_df["signed_accel"] <= decel_threshold	This line marks rows that qualify as hard breaking event as defined by the previous line
decel_by_agent = main_df.groupby(["id", "report_group"]).agg(hard_decel_events = ("hard_decel_flag", "sum"), mean_speed = ("speed_total", "mean"), mean_signed_accel = ("signed_accel", "mean")).reset_index()	This block groups the data by agent and report group, then calculates the total number of hard deceleration events, mean speed, and acceleration for each agent.
decel_by_group = decel_by_agent.groupby("report_group").agg(agents_in_group = ("id", "nunique"), total_hard_decel_events = ("hard_decel_events", "sum"), mean_hard_decel_events_per_agent = ("hard_decel_events", "mean"), median_hard_decel_events_per_agent = ("hard_decel_events", "median")).reset_index()	This block aggregates the agent-level deceleration data, and calculates the total and average number of hard deceleration events per agent for each report group.
print(decel_by_group) decel_by_agent.to_csv(decel_agent_file, index=False) decel_by_group.to_csv(decel_group_file, index=False)	These lines print the deceleration summary to the console and save both the agent and group deceleration data to their respective CSV files.
plt.figure(figsize=(9, 6)) plt.bar(decel_by_group["report_group"], decel_by_group["mean_hard_decel_events_per_agent"])	These two lines create a bar chart comparing the mean number of hard deceleration events per agent to each report group.
plt.title("Mean Hard Deceleration Events per Agent") plt.xlabel("Agent Group") plt.ylabel("Mean Hard Deceleration Events") plt.grid(axis="y", alpha=0.3)	These lines give the deceleration bar chart a title, axes labels and a semi transparent grid
plt.tight_layout() plt.show()	These two lines clean up the figure's spacing and then print the chart
fastest_group = group_summary.loc[group_summary["mean_speed"].idxmax(), "report_group"] slowest_group = group_summary.loc[group_summary["mean_speed"].idxmin(), "report_group"]	These two lines identify the groups with the highest and lowest mean speeds by locating rows in the group summary

most_decel_group = decel_by_group.loc[decel_by_group["mean_hard_decel_events_per_agent"].idxmax(), "report_group"] least_decel_group = decel_by_group.loc[decel_by_group["mean_hard_decel_events_per_agent"].idxmin(), "report_group"]	These two lines identify the groups with the highest and lowest average hard deceleration events per agent by locating rows in the deceleration summary.
print("Fastest average group:", fastest_group) print("Slowest average group:", slowest_group)	These two lines print the groups with the fastest and slowest mean speed
print("Most hard deceleration events per agent:", most_decel_group) print("Fewest hard deceleration events per agent:", least_decel_group)	These two lines print the groups with the most and fewest average hard deceleration events per agent
av_df = main_df[main_df["report_group"] == "AV"].sort_values("time")	This line filters the DataFrame down to only AV agents and sorts by time, for AV data analysis
plt.figure(figsize=(12, 5)) plt.plot(av_df["time"], av_df["speed_total"], color="blue", linewidth=1.5, label="AV Speed")	These two lines create a AV speed over time plot to visualize AV movement
plt.axhline(av_df["speed_total"].mean(), linestyle="--", color="gray", label=f"Mean = {av_df['speed_total'].mean():.2f} m/s")	This line adds a dashed grey line that is the mean AV speed to the plot
plt.title("AV Speed Over Time") plt.xlabel("Time (s)") plt.ylabel("Speed (m/s)") plt.legend() plt.grid(alpha=0.3)	These lines give the previously made plog a title, axes labels and a semi transparent grid
plt.tight_layout() plt.show()	These two lines clean up the figure's spacing and then print the chart
plt.figure(figsize=(12, 5)) plt.plot(av_df["time"], av_df["signed_accel"], color="red", linewidth=1, label="AV Signed Accel")	These two lines create a AV signed acceleration over time plot, creating a visualization of the AV's acceleration
plt.axhline(0, color="black", linewidth=0.8, linestyle="--") plt.axhline(decel_threshold, color="orange", linewidth=1, linestyle=":", label=f"Hard decel threshold ({decel_threshold} m/s\u00b2)")	These lines add two horizontal reference lines to the plot, one to distinguish acceleration from deceleration, and one at the hard deceleration threshold for visual reference.

plt.title("AV Signed Acceleration Over Time") plt.xlabel("Time (s)") plt.ylabel("Signed Acceleration (m/s\u00b2)") plt.legend() plt.grid(alpha=0.3)	These lines give the previously made plog a title, axes labels and a semi transparent grid
plt.tight_layout() plt.show()	These two lines clean up the figure's spacing and then print the chart
with open(polygon_file, "r", encoding="utf-8") as f: nb = nbformat.read(f, as_version=4)	These two lines open the polygon Jupyter Notebook file and read its contents into a notebook object using the nbformat library
all_code = "\n".join(cell.source for cell in nb.cells if cell.cell_type == "code")	This line extracts and joins all code cell contents from the notebook into a single string
match = re.search(r"polygons\s*=\s*(\{.*?\})", all_code, re.S)	This line searches the extracted notebook code for a the variable "polygons" using a regular expression, capturing its contents for use
if match is None: raise ValueError("Could not find polygons dictionary in Polygons.ipynb")	This line raises an error and halts execution if the polygon dictionary was not found in the notebook, preventing the program from continuing with missing data.
polygons = ast.literal_eval(match.group(1))	This line converts the extracted polygon dictionary string into a Python dictionary object using ast.literal_eval.
print(type(polygons)) print(list(polygons.keys())[:10])	These two lines print the type of the polygons object to confirm it was parsed correctly, and display the first ten polygon

fig, ax = plt.subplots(figsize=(10, 10))
 	
color_map = {
  "AV": "blue",
  "Non-AV Passenger Vehicle": "red",
  "Pedestrian": "green"}
	These two lines create a figure with axes for plotting and define a color map for the three main groups

for poly_id, coords in polygons.items():
  patch = MplPolygon(coords, closed=True, fill=False, alpha=0.4)
  ax.add_patch(patch)
	This loops through each polygon, creates a hollow transparent polygon shape from its coordinates, and adds it to the plot axes.

sample_ids = []
for group_name in ["AV", "Non-AV Passenger Vehicle", "Pedestrian"]:
  group_ids = main_df.loc[main_df["report_group"] == group_name, "id"].drop_duplicates()
  ids_here = group_ids.sample(n=min(5, len(group_ids)), random_state=42).tolist()
  sample_ids.extend(ids_here)
	These build a sample list of agent IDs by randomly selecting up to five agents from each group, for a reproducible sample

plot_df = main_df[main_df["id"].isin(sample_ids)].copy()	This line filters the main DataFrame down to only the sampled agent IDs, to use for the trajectory plot

for agent_id in plot_df["id"].unique():
  temp = plot_df[plot_df["id"] == agent_id].sort_values("time")
  group_name = temp["report_group"].iloc[0]
  ax.plot(temp["xloc_kf"], temp["yloc_kf"], color=color_map[group_name], linewidth=1)
	This loops through each sampled agent, uses .sort_values to sort their data by time, and plots their x and y trajectory on the map using the color assigned

for group_name, color in color_map.items(): ax.plot([], [], color=color, label=group_name)	This loop adds a placeholder line for each group to populate the legend with the correct color and label
ax.legend() ax.set_title("Sample Agent Movement Through the Intersection") ax.set_xlabel("X Location (m)") ax.set_ylabel("Y Location (m)") ax.set_aspect("equal")	These create a legend, add plot and axes titles and sets the aspect ratio with .set_aspect()
plt.tight_layout() plt.show()	These two lines clean up the figure's spacing and then print the chart
times = sorted(main_df["time"].unique())[::5] fig2, ax2 = plt.subplots(figsize=(10, 10))	These two lines make a list of every fifth unique timestamp for animation frames, then create a new square figure and axes to render the animation on

for poly_id, coords in polygons.items():
  patch = MplPolygon(coords, closed=True, fill=False, alpha=0.3, edgecolor="gray")
  ax2.add_patch(patch)
	This loops through each polygon and draws it on the animation figure as a hollow grey transparent shape to

	serve as a background for the animation
ax2.set_xlim(main_df["xloc_kf"].min() - 5, main_df["xloc_kf"].max() + 5) ax2.set_ylim(main_df["yloc_kf"].min() - 5, main_df["yloc_kf"].max() + 5) ax2.set_aspect("equal") ax2.set_xlabel("X Location (m)") ax2.set_ylabel("Y Location (m)") title_text = ax2.set_title("")	These lines set the x and y axis limits with a small buffer, sets an equal aspect ratio, adds axes labels, and sets a blank title to be updated.
for group_name, color in color_map.items(): ax2.plot([], [], color=color, label=group_name, marker="o", linestyle="None") ax2.legend(loc="upper right")	This adds placeholder markers to the animation figure to populate the legend with each report group's color, then places the legend in the upper right corner.
scat = ax2.scatter([], [], s=20, zorder=5)	This line creates an empty scatter plot object on the animation axes to be updated each frame to display agent positions

def update(t):
  frame = main_df[main_df["time"] == t]
  scat.set_offsets(frame[["xloc_kf", "yloc_kf"]].values)
  scat.set_color([color_map[g] for g in frame["report_group"]])
  scat.set_sizes([60 if g == "AV" else 15 for g in frame["report_group"]])
  title_text.set_text(f"Agent Positions at t = {t:.1f} s")
  return scat, title_text
	This function defines how each animation frame is rendered by filtering the data to the current timestamp, then updating the scatter plot with agent positions, colors, and sizes, with AV agents displayed larger than others with .set_offsets, set.color, .set_sizes, .set_text

ani = animation.FuncAnimation(fig2, update, frames=times[:200], interval=100, blit=False) ani.save("agent_animation.gif", writer="pillow", fps=10, dpi=80)	These two lines create the animation by calling the update function for each of the 200 timestamps at 100ms intervals, then save it as a GIF file using the Pillow library at 10 frames per second.
print("Saved agent_animation.gif") plt.close(fig2) HTML(ani.to_jshtml())	These lines confirm the GIF was saved, close the animation figure, and render the animation as an interactive HTML widget